

SOC Components - Processor

Dr. Robin Singla

PROCESSOR SELECTION FOR SOC

- Year by year, SOC processors and systems are becoming more complex. The SOC designer will be dealing with increasingly complex processors.

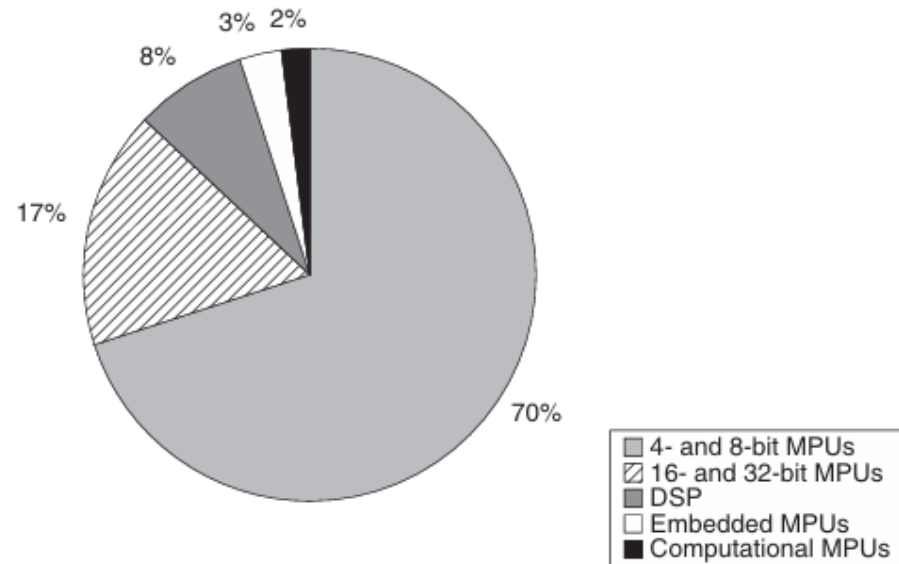


Figure 3.1 Worldwide production of microprocessors and controllers [227].

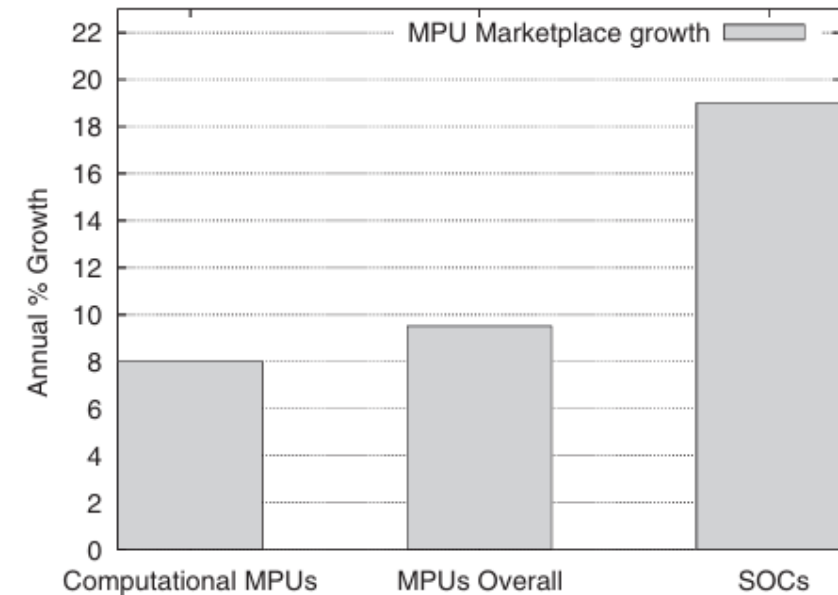


Figure 3.2 Annual growth in demand for microprocessors and controllers [227].

PROCESSOR SELECTION FOR SOC

- Especially in SOC-type applications, the processor itself is a small component occupying just a few percent of the die.
- SOC designs often use many different types of processors suited to the application.
- Often, noncritical processors are acquired (purchased) as design files (IP) and are integrated into the SOC design.
- Therefore, a specialized SOC design may integrate generic processor cores designed by other parties.
- Table illustrates the relative advantages of different approaches to using intellectual property in SOC designs.

TABLE 3.1 Optimized Designs Provide Better Area–Time Performance at the Expense of Design Time

Type of Design	Design Level	Relative Expected Area × Time
Customized hard IP	Complete physical	1.0
Synthesized firm IP	Generic physical	3.0–10.0
Soft IP	RTL or ASIC	10.0–100.0

PROCESSOR SELECTION FOR SOC

- **For many SOC design situations, the selection of the processor is the most obvious task and, in some ways, the most restricted.**
- **The processor must run a specific system software, so at least a core processor — usually a general-purpose processor (GPP) — must be selected for this function.**
- **In computelimited applications, the primary initial design thrust is to ensure that the system includes a processor configured and parameterized to meet this requirement.**
- **In some cases, it may be possible to merge these processors, but that is usually an optimization consideration dealt with later.**
- **In determining the processor performance and the system performance, we treat memory and interconnect components as simple delay elements**

PROCESSOR SELECTION FOR SOC

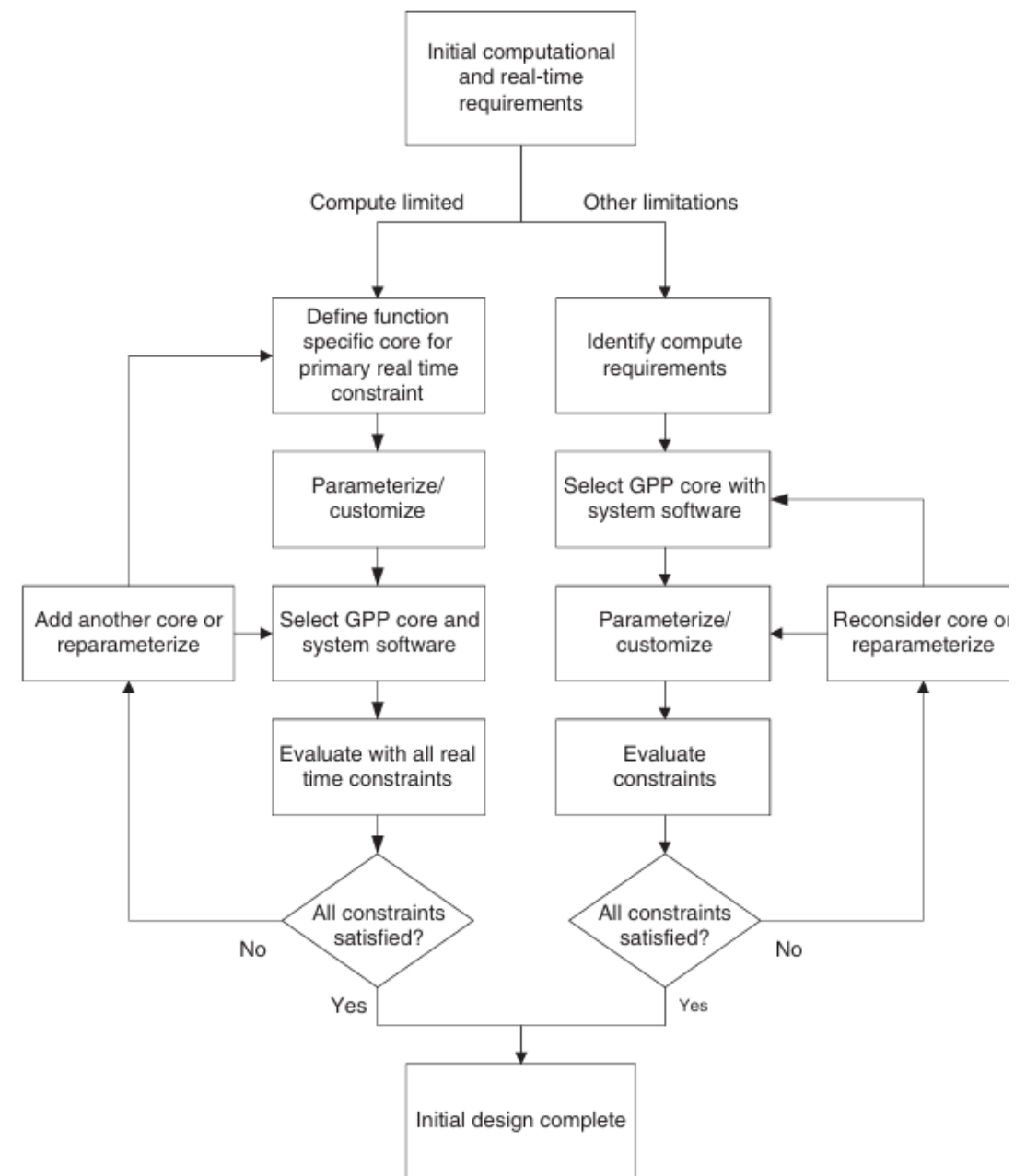


Figure 3.4 Process of processor core selection.

PROCESSOR SELECTION FOR SOC - **Soft Processors**

- **The term “ soft core ” refers to an instruction processor design in a bitstream format that can be used to program a field programmable gate array (FPGA) device.**
- **The 4 main reasons for using such designs, despite their large area power–time cost, are**
 - 1. cost reduction in terms of system-level integration,**
 - 2. design reuse in cases where multiple designs are really just variations on one,**
 - 3. creating an exact fit for a microcontroller/peripheral combination, and**
 - 4. Providing future protection against discontinued microcontroller variants.**

PROCESSOR SELECTION FOR SOC - **Soft Processors**

TABLE 3.2 Some Features of Soft-Core Processors

	Nios II (fast) [13]	MicroBlaze [260]	OpenRISC [190]	Leon4 [106]
Open source	No	No	Yes	Yes
Hardware FPU	Yes	Yes	No	Yes
Bus standard	Avalon	CoreConnect	WISHBONE	AMBA
Integer division unit	Yes	Yes	No	Yes
Custom coprocessors/ instructions	Yes	Yes	Yes	Yes
Maximum frequency on FPGA (MHz)	290	200	47	125
Max MIPS on FPGA	340	280	47	210
Resources	1800 LE	1650 slices	2900 slices	4000 slices
Area estimate	1500 A	800 A	1400 A	1900 A

BASIC CONCEPTS IN PROCESSOR ARCHITECTURE

- **The processor architecture consists of the instruction set of the processor.**
- **While the instruction set implies many implementations (microarchitecture) details, the resulting implementation is a great deal more than the instruction set.**
- **It is the synthesis of the physical device limitations with area – time – power trade-offs to optimize specified user requirements.**

BASIC CONCEPTS IN PROCESSOR ARCHITECTURE

Instruction Set

- **The instruction set for most processors is based upon a register set to hold operands and addresses.**
- **The register set size varies from 8 to 64 words or more, each word consisting of 32 – 64 bits.**
- **An additional set of floating-point registers (32 – 128 bits) is usually also available.**
- **A typical instruction set specifies a program status word, which consists of various types of control status information, including condition codes (CCs) set by the instruction.**
- **Common instruction sets can be classified by format differences into two basic types, the load–store (L/S) architecture and the register–memory (R/M) architecture:**

BASIC CONCEPTS IN PROCESSOR ARCHITECTURE

Instruction Set

- **The L/S instruction** set includes the RISC microprocessors.
 - Arguments must be in registers before execution.
 - An ALU instruction has both source operands and result specified as registers.
 - The advantages of the L/S architecture are regularity of execution and ease of instruction decode.
 - A simple instruction set with straightforward timing is easily implemented.
- **The R/M architectures** include instructions that operate on operands in registers or with one of the operands in memory.
 - In the R/M architecture, an ADD instruction might sum a register value and a value contained in memory, with the result going to a register.
 - The R/M instruction sets trace their evolution to the IBM mainframes and the Intel x86 series (now called the Intel IA32).

BASIC CONCEPTS IN PROCESSOR ARCHITECTURE

Instruction Set

- **The trade-off in instruction sets is an area–time compromise.**
- **The R/M approach offers a more concise program representation using fewer instructions of variable size compared with L/S.**
- **Programs occupy less space in memory and require smaller instruction caches.**
- **The variable instruction size makes decoding more difficult.**
- **The decoding of multiple instructions requires predicting the starting point of each.**
- **The R/M processors require more circuitry (and area) to be devoted to instruction fetch and decode.**

BASIC CONCEPTS IN PROCESSOR ARCHITECTURE

Instruction Set

TABLE 3.3 Instruction Set Mnemonic Operations

Mnemonic	Operation
ADD	Addition
SUB	Subtraction
MPY	Multiplication
DIV	Division
CMP	Compare
LD	Load (a register from memory)
ST	Store (a register to memory)
LDM	Load multiple registers
STM	Store multiple registers
MOVE	Move (register to register or memory to memory)
SHL	Shift left
SHR	Shift right
BR	Unconditional branch
BC	Conditional branch
BAL	Branch and link

BASIC CONCEPTS IN PROCESSOR ARCHITECTURE

Instruction Set

TABLE 3.4 Data-Type Modifiers (OP.modifier)

Modifier	Data Type
B	Byte (8 bits)
H	Halfword (16 bits)
W	Word (32 bits)
F	Floating point (32 bits)
D	Double-precision floating point (64 bits)
C	Character or decimal in an 8-bit format

BASIC CONCEPTS IN PROCESSOR ARCHITECTURE

Branches

- **Branches (or jumps) manage program control flow.**
- **They typically consist of unconditional BR, conditional BC, and subroutine call and return (link).**
- **The BC tests the state of the CC, which usually consists of 4 bits in a program status or control register.**
- **Typically, the CC is set by an ALU instruction to record one of several results (encoded in 2 or 4 bits), for example, specifying whether the instruction has generated**
 - 1. a positive result,**
 - 2. a negative result,**
 - 3. a zero result, or**
 - 4. an overflow.**

BASIC CONCEPTS IN PROCESSOR ARCHITECTURE

Interrupts and Exceptions

- Many embedded SOC controllers have external interrupts and internal exceptions, which indicate the need for attention by an interrupt manager (or handler) program.
- These facilities can be managed and supported in various ways:
- **User Requested versus Coerced.** The former often covers erroneous execution, such as divide by zero, while the latter is usually triggered by external events, such as device failure.
- **Maskable versus Nonmaskable.** The former type of event can be ignored by setting a bit in an interrupt mask, while the latter cannot be ignored.

BASIC CONCEPTS IN PROCESSOR ARCHITECTURE

Interrupts and Exceptions

- **Terminate versus Resume.** An event such as divide by zero would terminate ordinary processing, while a processor resumes operation.
- **Asynchronous versus Synchronous.** Interrupt events can occur in asynchrony with the processor clock by an external agent or not, as when caused by a program 's execution.
- **Between versus Within Instructions.** Interrupt events can be recognized only between instructions or within an instruction execution.

BASIC CONCEPTS IN PROCESSOR MICROARCHITECTURE

- **Almost all modern processors use an instruction execution pipeline design.**
- **Simple processors issue only one instruction for each cycle; others issue many.**
- **Many embedded and some signal processors use a simple issue - one – instruction per - cycle design approach.**
- **But the bulk of modern desktop, laptop, and server systems issue multiple instructions for each cycle.**

BASIC CONCEPTS IN PROCESSOR MICROARCHITECTURE

- Every processor has a memory system, execution unit (data paths), and instruction unit.
- The faster the cache and memory, the smaller the number of cycles required for fetching instructions and data (IF and DF).
- The more extensive the execution unit, the smaller the number of execution cycles (EX).
- The control of the cache and execution unit is done by the instruction unit.

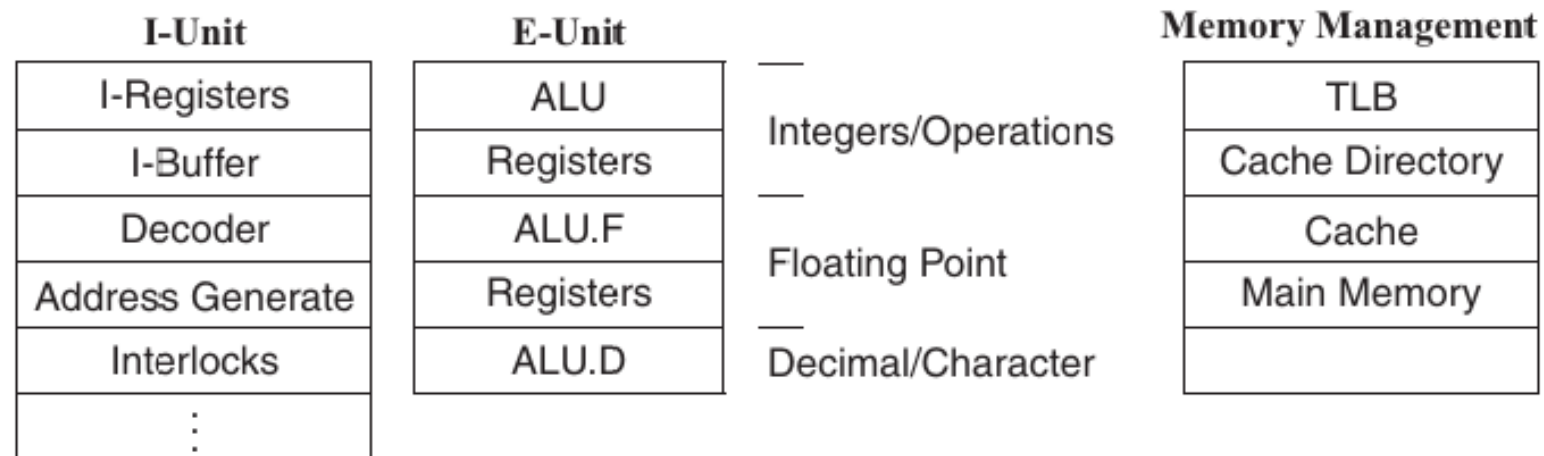


Figure 3.7 Processor units.

BASIC CONCEPTS IN PROCESSOR **MICROARCHITECTURE**

- The pipeline mechanism or control has many possibilities.
- Potentially, it can execute one or more instructions for each cycle.
- Instructions may or may not be decoded and/or executed in program order.
- Indeed, instructions from several different programs can be executed in the same cycle in multithreaded pipelines.
- Regardless of the type of pipeline, “ breaks ” or delays are the major limit on performance.
- Pipeline delays or breaks generally arise from
 1. **Data Conflicts** - Unavailability of a Source Operand. This can occur for several reasons; typically, the current instruction requires an operand that is the result of a preceding uncompleted instruction. Extensive operand buffering can minimize this effect.

BASIC CONCEPTS IN PROCESSOR MICROARCHITECTURE

2. **Resource Contention-** Multiple successive instructions use the same resource or an instruction with a long execution time delays a successor instruction's execution. Additional resources (floating-point units, register ports, and out-of-order execution) help reduce contention.
3. **Run - On Delays (in Order Execution Only)-** When instructions must complete the WB (writeback) in program order, any delay in execution (as in the case of multiply or divide) necessarily delays the instruction execution in the pipeline.
4. **Branches -** The pipeline is delayed by branch resolution and/or by the delay in fetching the branch target instruction before the pipeline can resume execution. Branch prediction, branch tables, and buffers can all be used to minimize the effect of branches.